

Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

nihal.altuntas@iuc.edu.tr

Yinelemeli Metotlar (Recursion)

- Recursion nedir?
  - Bir metot'un kendini çağırması?

- Örnek: Tavşan çiftliği
  - Elinizde bir çift yavru tavşan var



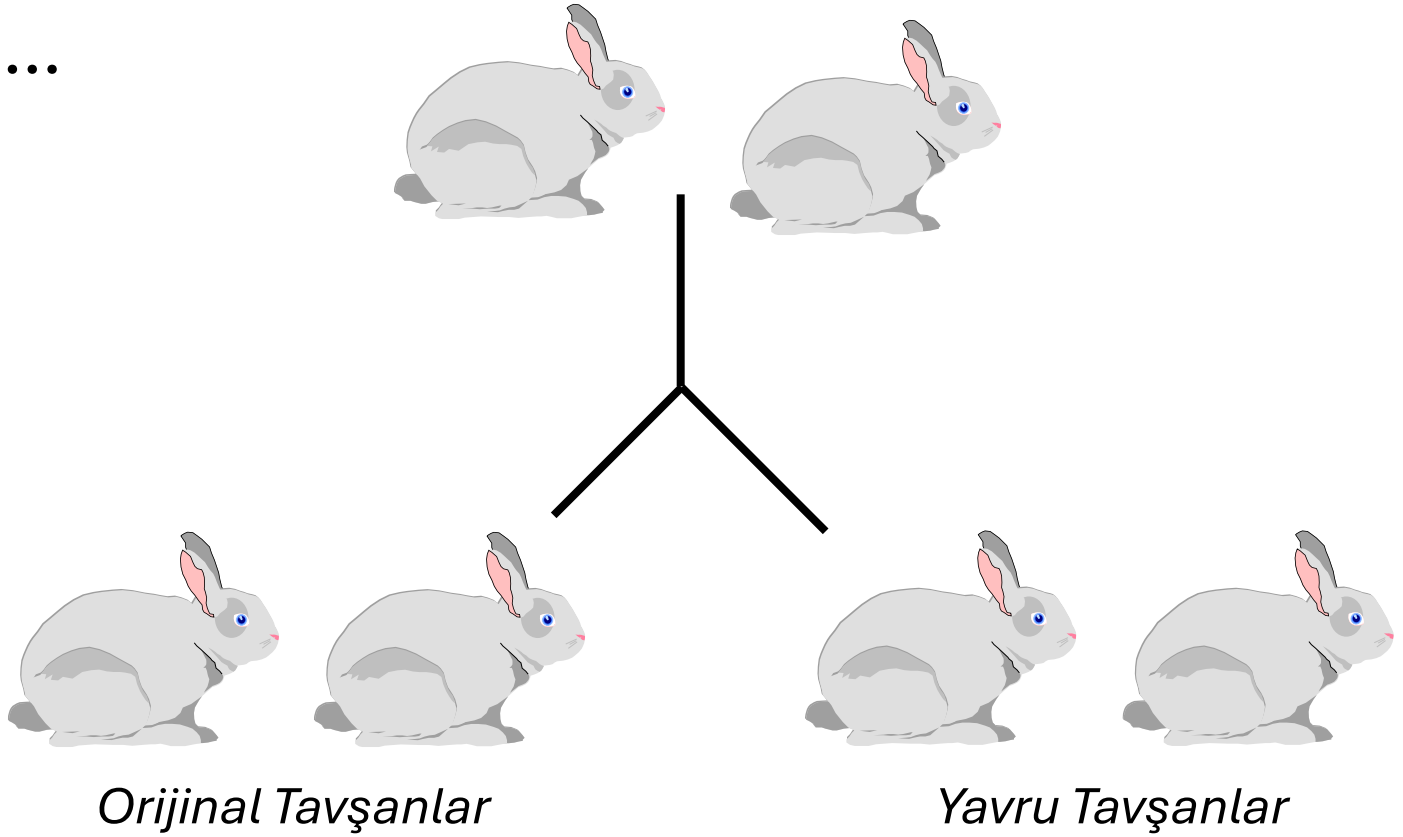
Yinelemeli Metotlar (Recursion)

- Tavşan çiftliği kuralları
  - Yetişin bir çift tavşanın her ay yeni yavruları oluyor
  - Bir çift tavşanın her zaman iki yavrusu oluyor
    - Bir dişi
    - Bir erkek
  - Yeni doğan tavşanların 2 ay sonra yavrusu olmaya başlıyor
  - Hiçbir tavşan ölmüyor

- Bir yıl sonra kaç tavşanınız olur?

Yinelemeli Metotlar (Recursion)

- İki Ay Sonra...



Yinelemeli Metotlar (Recursion)

Başlangıç



1. Ayın Sonu



2. Ayın Sonu



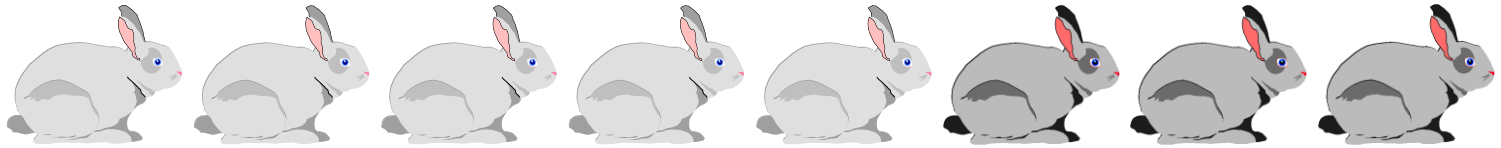
3. Ayın Sonu



4. Ayın Sonu



5. Ayın Sonu



ANAHTAR



= Yetişkin bir çift



= Yavru bir çift

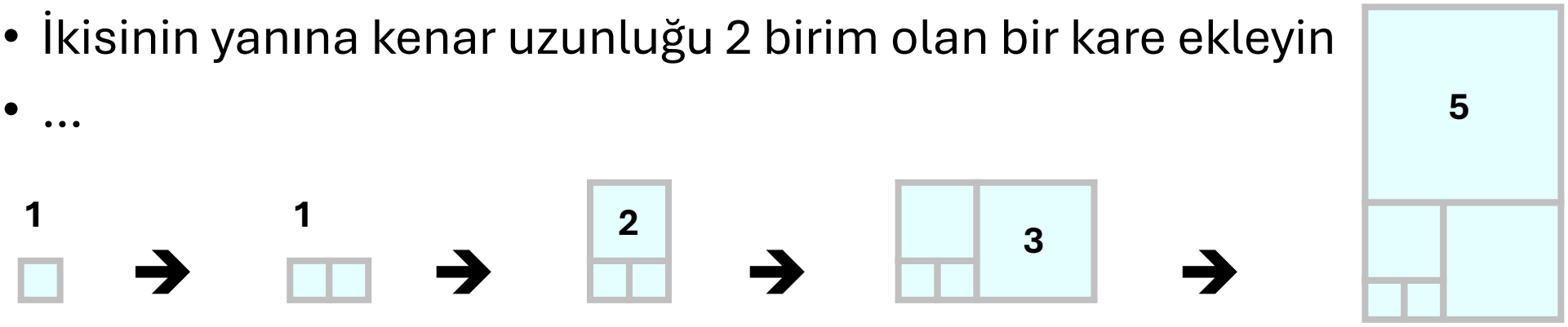
Yinelemeli Metotlar (Recursion)

Zaman	Yetişkin	Yavru	Toplam
Başlangıç	0	1	1
1. Ayın Sonu	1	0	1
2. Ayın Sonu	1	1	2
3. Ayın Sonu	2	1	3
4. Ayın Sonu	3	2	5
5. Ayın Sonu	5	3	8
6. Ayın Sonu	8	5	13
7. Ayın Sonu	13	8	21
8. Ayın Sonu	21	13	34
9. Ayın Sonu	34	21	55
10. Ayın Sonu	55	34	89
11. Ayın Sonu	89	55	144
12. Ayın Sonu	144	89	233

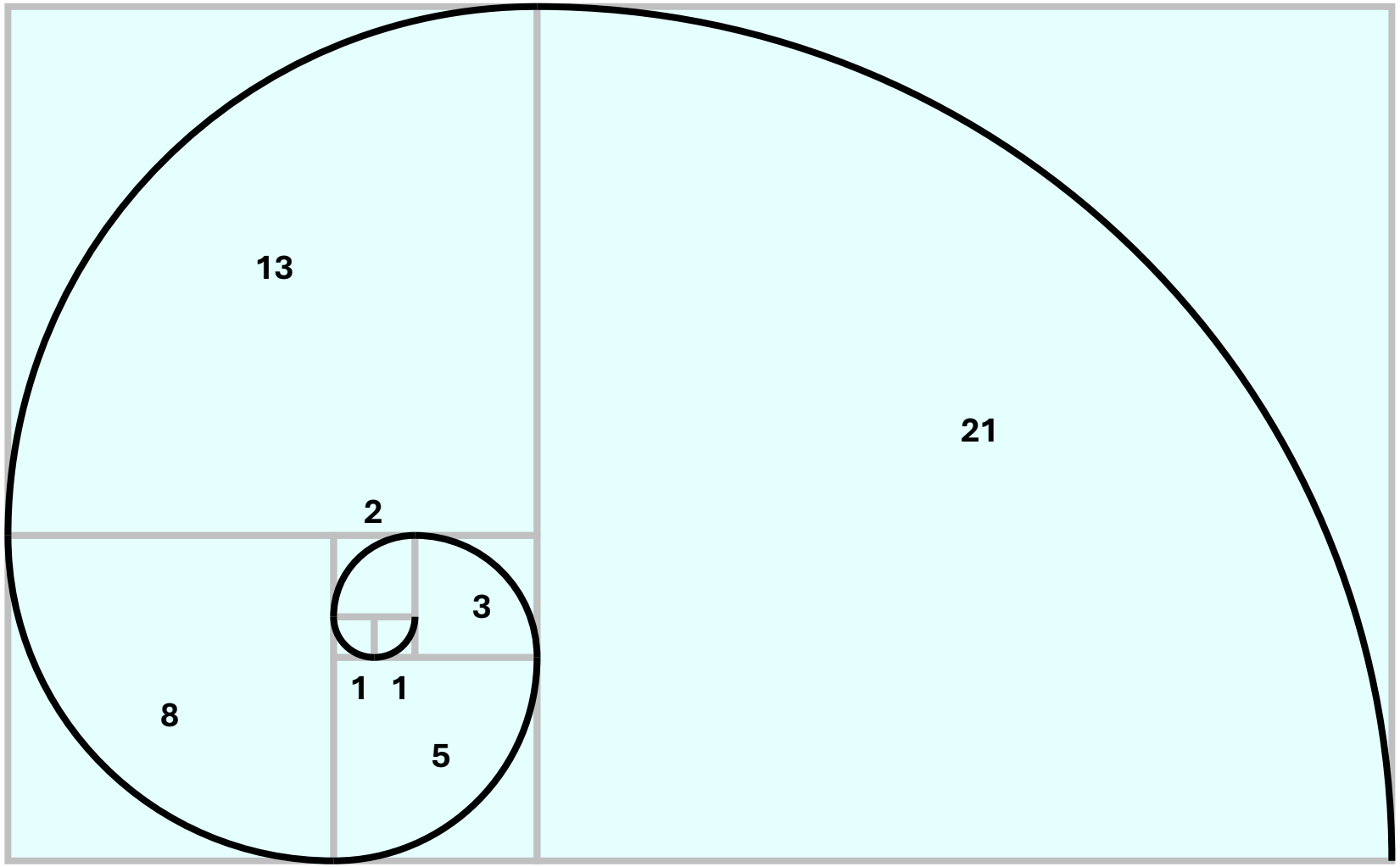
Örüntü gören?

Yinelemeli Metotlar (Recursion)

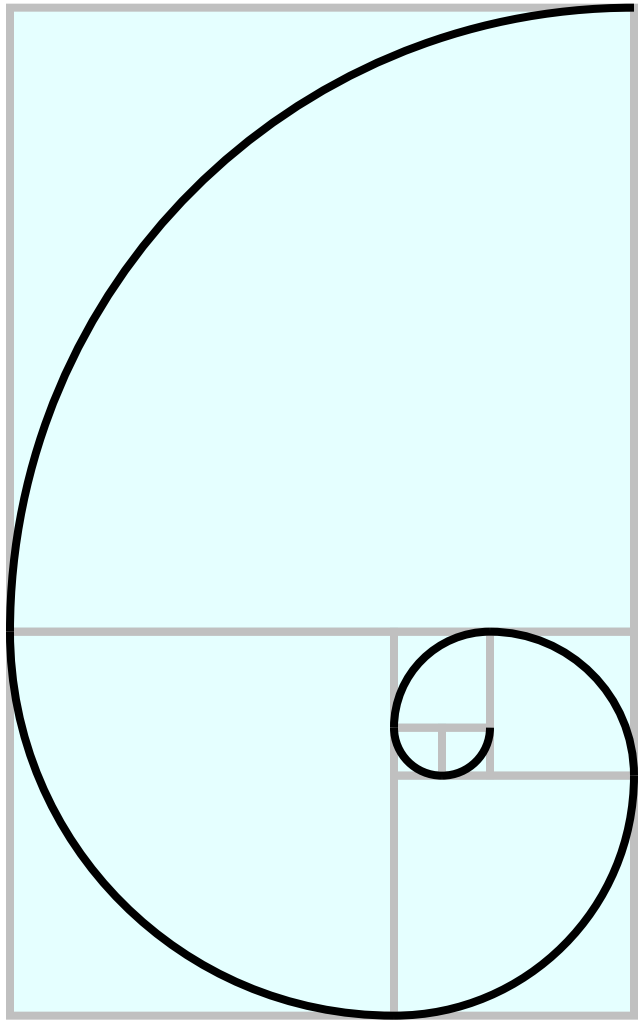
- Tablodaki Toplam sütunu:
  - 1, 1, 2, 3, 5, 8, 13, 21, …
- Başka bir örnek
  - Kenar uzunluğu 1 birim olan bir kare çizin
  - Yanına kenar uzunluğu 1 birim olan bir kare daha ekleyin
  - İkisinin yanına kenar uzunluğu 2 birim olan bir kare ekleyin
  - …

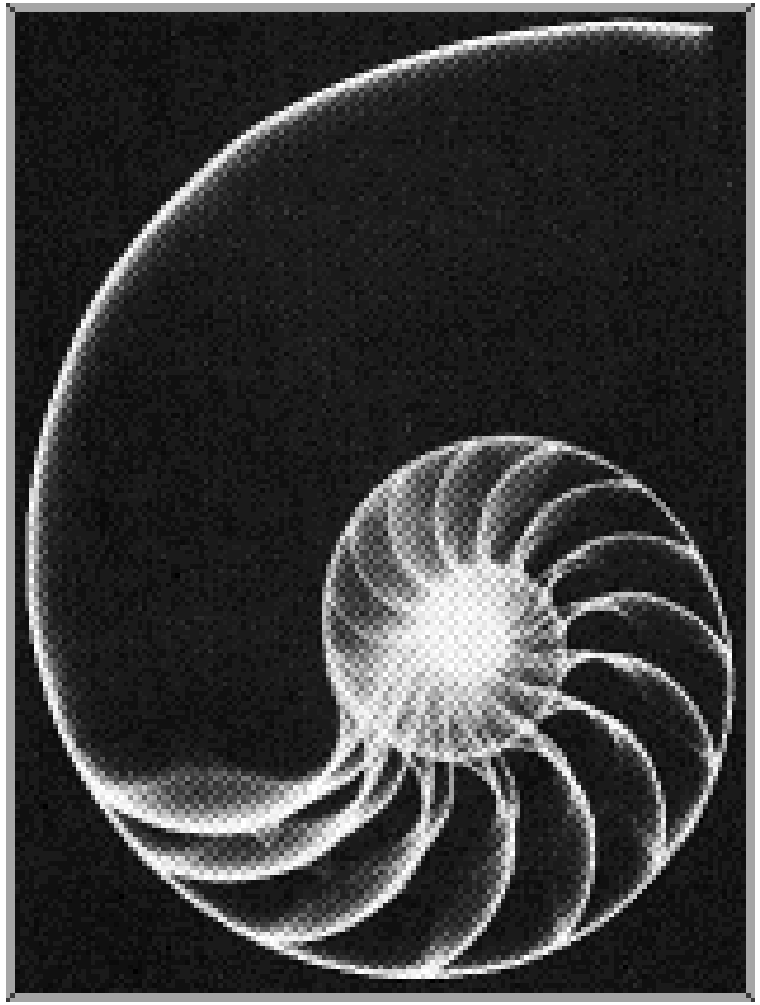


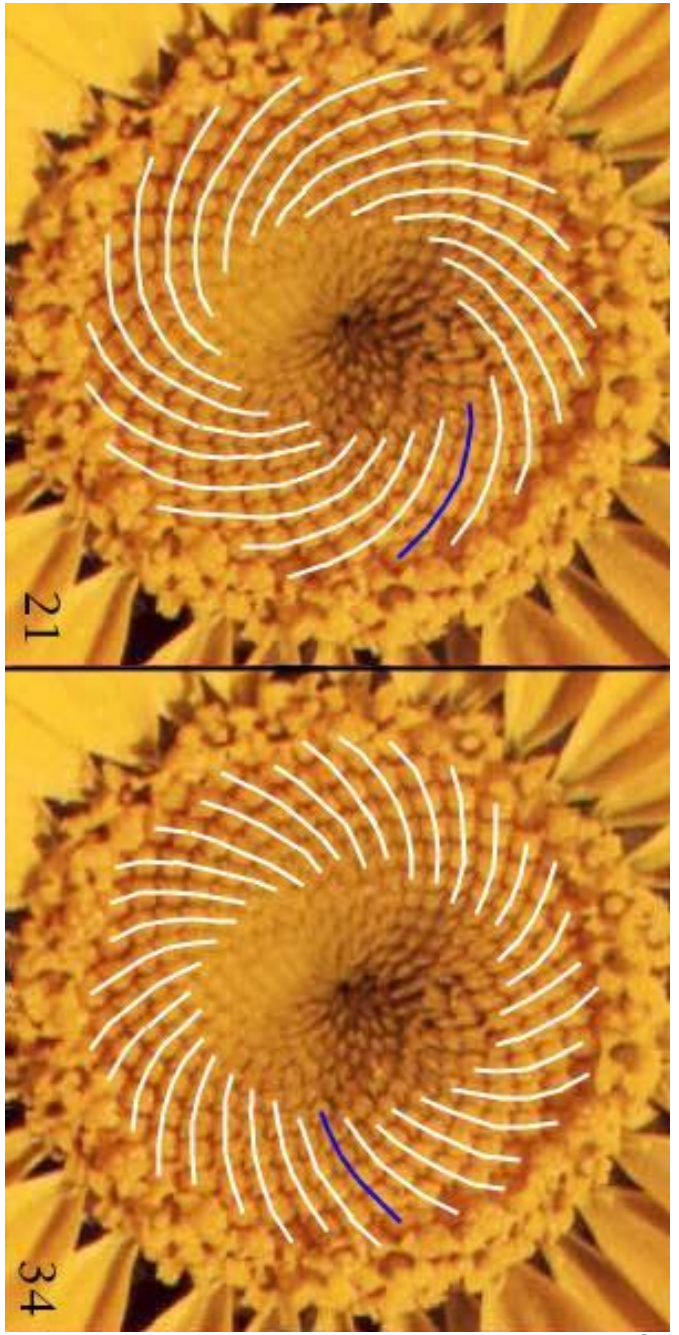
Yinelemeli Metotlar (Recursion)



Yinelemeli Metotlar (Recursion)







Yinelemeli Metotlar (Recursion)

- Fibonacci Serisi
  - 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, … ,

$$Fib(n) = \left[\frac{1+\sqrt{5}}{2}\right]^n - \left[\frac{1-\sqrt{5}}{2}\right]^n$$

Fib(n) = Fib(n-1) + Fib(n-2);
Fib(0) = 0;
Fib(1) = 1;

- Verilen sıradaki sayı kendisinden önce gelen iki sayının toplamına eşittir

Yinelemeli Metotlar (Recursion)

- Fibonacci Serisi
  - Yinelemeli olarak tanımlanabilir
  - Belirli bir durma koşulu vardır
  - Belirlenebilir ve hesaplanabilir (toplama)

Fib(n) = Fib(n-1) + Fib(n-2);
Fib(0) = 0;
Fib(1) = 1;

Yinelemeli Metotlar (Recursion)

Başlangıç olarak ne biliyoruz?
Belirli bir konumdaki bir sayı için
Fibonacci değerini döndüren
bir fonksiyona
ihtiyacımız olduğunu biliyoruz.

Bu durumda
parametre olarak tamsayı alan
ve tamsayı döndüren bir fonksiyon
tanımlamamız gerekir

Yinelemeli Metotlar (Recursion)

```
def fib(num):
```

Yinelemeli bir fonksiyonla yaptığımız İLK şey nedir?

Nasıl sona ereceğini planlarız!

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
```

Fibonacci serisi için
özel bir durum biliyoruz:

F(0) = 0

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
    elif num == 1:
        return 1
```

Yinelememizi sonlandırabilecek ikinci bir özel durum da biliyoruz:

F(1) = 1.

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
    elif num == 1:
        return 1
    else:
        return fib(num - 1) + fib(num - 2)
```

Formülümüzün son kısmı sadece:

F(n) = F(n-1) + F(n-2)

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
    elif num == 1:
        return 1
    else:
        return fib(num - 1) + fib(num - 2)
```

Bu güvenli mi?
Birisi fonksiyona
parametre olarak 0 verirse?

Ne olur?

Peki parametre 1 olursa?

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
    elif num == 1:
        return 1
    else:
        return fib(num - 1) + fib(num - 2)

if __name__ == "__main__":
    for i in range(10):
        print(fib(i))
```

Bu fonksiyonu test etmek için bir main yazmak bizim sorumluluğumuzda.

Yinelemeli Metotlar (Recursion)

```
def fib(num):
    if num == 0:
        return 0
    elif num == 1:
        return 1
    else:
        return fib(num - 1) + fib(num - 2)

if __name__ == "__main__":
    for i in range(10):
        print(fib(i))
```

Tamam mıyız?
Peki ya negatif sayılar?
Daha fazla çalışmaya ihtiyaç var

Yinelemeli Metotlar (Recursion)

- Bir örnek daha
  - pow(x, y) = x^y

$x^y = x * x^{y-1}$

$x^1 = x;$

$x^0 = 1;$

Yinelemeli Metotlar (Recursion)

- Bir örnek daha
  - pow(x, y) = x^y

```
def pow(base, exponent):
    if exponent == 0:
        return 1
    elif exponent == 1:
        return base
    else:
        exponent -= 1
        return base * pow(base, exponent)
```

Yinelemeli Metotlar (Recursion)

- pow(x, y) = x^y

```
def pow(base, exponent):
    if exponent == 0:
        return 1
    elif exponent == 1:
        return base
    else:
        exponent -= 1
        return base * pow(base, exponent)
```

- Nasıl döngüden daha hızlı olur?
  - Yerine for döngüsü kullansam?

Yinelemeli Metotlar (Recursion)

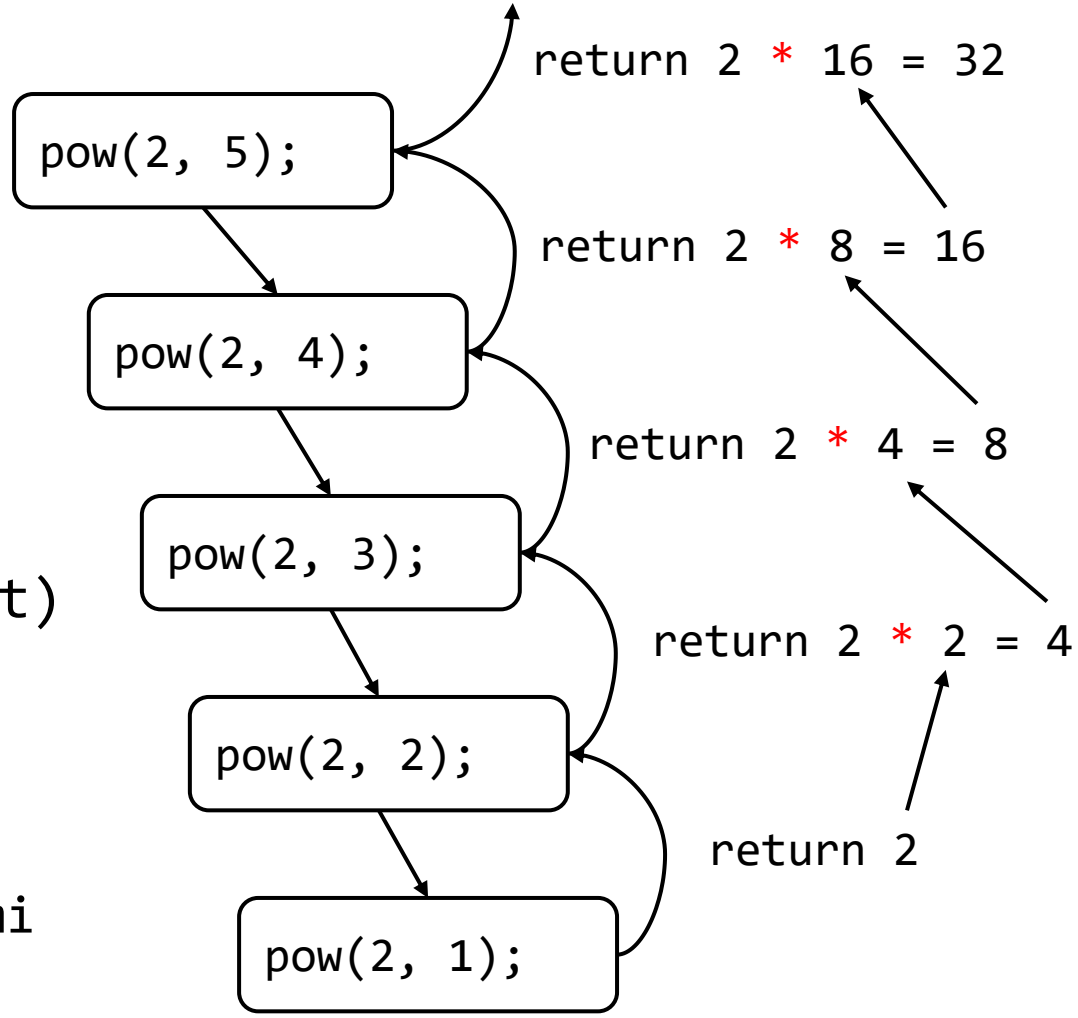
```
def pow_v2(base, exponent):
    result = 1
    for i in range(exponent):
        result *= base
    return result
```

```
pow_v2(2, 5);  ⟶  result = 1 * 2 * 2 * 2 * 2 * 2 = 32
```

- Örnek Çalıştırma
  - pow_v2(2, 5) hesabı için 5 çarpma işlemi
  - pow_v2(b, ***N***) hesabı için ***N*** çarpma işlemi

Yinelemeli Metotlar (Recursion)

```
def pow(base, exponent):
    if exponent == 0:
        return 1
    elif exponent == 1:
        return base
    else:
        exponent -= 1
        return base * pow(base, exponent)
```



- Örnek Çalıştırma
  - pow(2, 5) hesabı için 4 çarpma işlemi
  - pow(b, ***N***) hesabı için ***N-1*** çarpma işlemi

İyileşme yeterli mi?

Yinelemeli Metotlar (Recursion)

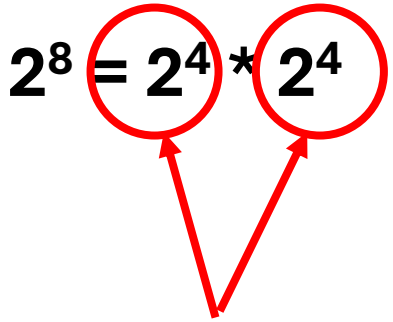
- $x^y = x * x^{y-1}$
- $x^1 = x;$
- $x^0 = 1;$

$$2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$$

Yinelemeli Metotlar (Recursion)

- $x^y = x * x^{y-1}$
- $x^1 = x;$
- $x^0 = 1;$

$$2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$$



Aynı oldukları için tekrar hesaplamaya gerek yok

Yinelemeli Metotlar (Recursion)

- Peki tek üstler için?
  - $x^{tek} = x * x^{tek-1}$

$2^{21} = 2 * 2^{20}$ (Tek sayı)

$2^{20} = 2^{10} * 2^{10}$

$2^{10} = 2^{5} * 2^{5}$

$2^{5} = 2 * 2^{4}$ (Tek sayı)

$2^{4} = 2^{2} * 2^{2}$

$2^{2} = 2^{1} * 2^{1}$

Yinelemeli Metotlar (Recursion)

```
def pow(base, exponent):
    if exponent == 0:
        return 1
    elif exponent == 1:
        return base
    elif exponent % 2 == 0:
        exponent //= 2
        half = pow(base, exponent)
        return half * half
    else:
        exponent -= 1
        oneless = pow(base, exponent)
        return oneless * base
```

Peki negatif üs değerleri?

Yinelemeli Metotlar (Recursion)

```
def pow(base, exponent):
    if exponent < 0:
        exponent *= -1
        return 1 / pow(base, exponent)
    elif exponent == 0:    return 1
    elif exponent == 1:    return base
    elif exponent % 2 == 0:
        exponent //= 2
        half = pow(base, exponent)
        return half * half
    else:
        exponent -= 1
        oneless = pow(base, exponent)
        return oneless * base
```

$$x^{-y} = \frac{1}{x^y}$$